

JDBC Inserts, Batch Processing and Transactions

Change the URL, username and password to match your own database.

Q1: Insert a single record and commit

Table:

```
CREATE TABLE playlist (
    id          INT PRIMARY KEY,
    song_name   VARCHAR(100),
    artist      VARCHAR(100)
);

// InsertPlaylistSong.java
// Inserts one song into the playlist table and commits the transaction.

import java.sql.*;

public class InsertPlaylistSong {
    private static final String URL  = "jdbc:mysql://localhost:3306/spotify_clone";
    private static final String USER = "root";
    private static final String PASS = "root123";

    public static void main(String[] args) {
        String sql = "INSERT INTO playlist (id, song_name, artist) VALUES (?, ?, ?)";
        Connection con = null;

        try {
            con = DriverManager.getConnection(URL, USER, PASS);
            con.setAutoCommit(false);          // start a transaction

            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setInt(1, 1);
                ps.setString(2, "Blinding Lights");
                ps.setString(3, "The Weeknd");
                ps.executeUpdate();
            }

            con.commit();                      // make the insert permanent
            System.out.println("Record inserted and committed.");

        } catch (SQLException e) {
            System.out.println("Error: " + e.getMessage());
            try { if (con != null) con.rollback(); } catch (SQLException ex) { ex.printStackTrace(); }
        } finally {
            try { if (con != null) con.close(); } catch (SQLException ex) { ex.printStackTrace(); }
        }
    }
}
```

Output:

Record inserted and committed.

Q2: addMultipleMovies() using batch processing

Table:

```
CREATE TABLE movies (
    movie_id    INT AUTO_INCREMENT PRIMARY KEY,
    movie_name  VARCHAR(100)
);

// MovieManager.java
// Inserts many movies at once using addBatch() and executeBatch().

import java.sql.*;
import java.util.Arrays;
import java.util.List;
```

```

public class MovieManager {
    private static final String URL = "jdbc:mysql://localhost:3306/bookmyshow_db";
    private static final String USER = "root";
    private static final String PASS = "root123";

    public static void addMultipleMovies(List<String> movieNames) {
        String sql = "INSERT INTO movies (movie_name) VALUES (?)";

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
            PreparedStatement ps = con.prepareStatement(sql)) {

            for (String name : movieNames) {
                ps.setString(1, name);
                ps.addBatch(); // add this insert to the batch
            }

            int[] results = ps.executeBatch(); // send all inserts together
            System.out.println(results.length + " movies inserted.");

        } catch (SQLException e) {
            System.out.println("Error inserting movies: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        addMultipleMovies(Arrays.asList("Inception", "Interstellar", "Dangal", "3 Idiots"));
    }
}

```

Output:

4 movies inserted.

Why batch is efficient: all the inserts are sent to the database together instead of one network round trip per row. For faster MySQL batching, add `?rewriteBatchedStatements=true` to the JDBC URL.

Q3: Shopping cart with transaction rollback

Table (product_id is the primary key, so duplicates fail):

```

CREATE TABLE orders (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2)
);

```

// CartOrderProcessor.java

// Inserts all cart products in one batch. If any insert fails, everything is rolled back.

```

import java.sql.*;

public class CartOrderProcessor {
    private static final String URL = "jdbc:mysql://localhost:3306/flipkart_db";
    private static final String USER = "root";
    private static final String PASS = "root123";

    public static void placeOrder(int[] ids, String[] names, double[] prices) {
        String sql = "INSERT INTO orders (product_id, product_name, price) VALUES (?, ?, ?)";
        Connection con = null;

        try {
            con = DriverManager.getConnection(URL, USER, PASS);
            con.setAutoCommit(false); // begin transaction

            try (PreparedStatement ps = con.prepareStatement(sql)) {
                for (int i = 0; i < ids.length; i++) {
                    ps.setInt(1, ids[i]);
                    ps.setString(2, names[i]);
                    ps.setDouble(3, prices[i]);
                    ps.addBatch();
                }
                ps.executeBatch(); // throws BatchUpdateException if any insert fails
            }
        }
    }
}

```

```

        con.commit(); // all inserts succeeded
        System.out.println("Order placed. All products saved.");
    } catch (SQLException e) { // BatchUpdateException is a SQLException
        System.out.println("Order failed: " + e.getMessage());
        try {
            if (con != null) {
                con.rollback(); // undo every insert in this transaction
                System.out.println("Transaction rolled back. No products were saved.");
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    } finally {
        try {
            if (con != null) {
                con.setAutoCommit(true); // restore default behaviour
                con.close();
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
}

public static void main(String[] args) {
    // Product ID 102 appears twice, so the batch fails and everything is rolled back
    int[] ids = {101, 102, 102, 103};
    String[] names = {"Phone", "Headphones", "Charger", "Case"};
    double[] prices = {15999.00, 1999.00, 799.00, 299.00};

    placeOrder(ids, names, prices);
}
}

```

Output (with the duplicate ID 102):

```

Order failed: Duplicate entry '102' for key 'orders.PRIMARY'
Transaction rolled back. No products were saved.

```

The exact message may differ slightly by MySQL version. After the rollback, `SELECT * FROM orders` shows no rows from this order, even though product 101 was inserted before the error.

Q4: Forgetting commit() when auto-commit is off

What happens to the data:

- With `setAutoCommit(false)`, the inserts run inside an open transaction. They are only temporary until `commit()` is called.
- Inside the same connection, a `SELECT` will show the new rows. Other connections and other users cannot see them, because of transaction isolation.
- If the program ends or the connection is closed without commit, MySQL rolls the transaction back, so all the inserted rows are lost and the table looks unchanged.
- While the transaction is open, the inserted rows stay locked, which can block other sessions that try to change the same rows.

Buggy code:

```

con.setAutoCommit(false);
ps.executeBatch();
// commit() is missing, so the data is lost when the connection closes
con.close();

```

Fix: call `commit()` after the inserts succeed, and roll back in the catch block if something fails.

```

Connection con = null;
try {
    con = DriverManager.getConnection(URL, USER, PASS);
    con.setAutoCommit(false);

```

```
ps.executeBatch();

    con.commit();                // FIX: saves the data permanently
} catch (SQLException e) {
    if (con != null) con.rollback(); // undo everything if something went wrong
    e.printStackTrace();
} finally {
    if (con != null) con.close();
}
```

Alternative: keep auto-commit on (the default), where each statement is committed automatically. Only turn it off when several statements must succeed or fail together.